МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение высшего образования

«ВЛАДИВОСТОКСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
(ФГБОУ ВО «ВВГУ»)

Институт информационных технологий и анализа данных
Кафедра информационных технологий и систем

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ

по дисциплине «Информатика и основы программирования»

Лабораторная работа 6. Реализация численных алгоритмов

Студент
гр. БИН-26-1 _____ И.Д. Плющиков

Преподаватель _____ М.В. Водяницкий

Владивосток 2026

1 Выполнение работы

1.1 Задание 1. Наибольший общий делитель

Постановка задачи: Пользователь вводит два целых числа a и b. Если хотя бы одно число отрицательное или оба равны нулю — вывести «Ошибка». Иначе найти НОД алгоритмом Евклида.

Изображение схемы для решения первого задания 1.

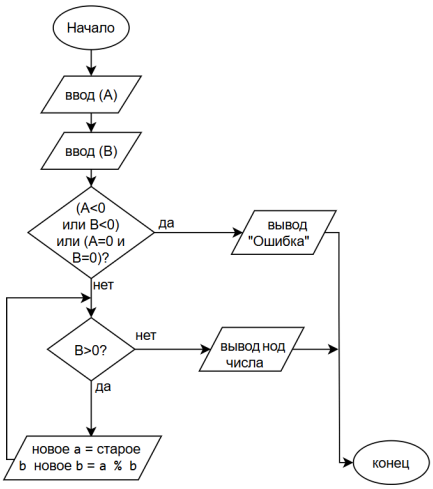


Рисунок 1 – схема программы для первого задания

Алгоритм: Пока $bn = 0$, заменить пару (a, b) на $(b, ab \bmod b)$. Когда $b = 0$, результат — a.

Изображение результата выполнения программы 2.

```
1  a = int(input())
2  b =int(input())
3  if (a<0 or b<0) or (a==0 and b==0):
4      print("Ошибка")
5  else:
6      while b>0:
7          a,b = b, a%b
8      print(f"НОД: {a}")
```

Рисунок 2 – код первого задания

Задание показывает классический алгоритм Евклида и обработку граничных случаев.

1.2 Задание 2. Проверка числа на простоту

Постановка задачи: Пользователь вводит целое число n. Вывести «Простое» или «Не простое». Числа меньше 2 считаются непростыми.

Изображение схемы для решения торого задания 3.

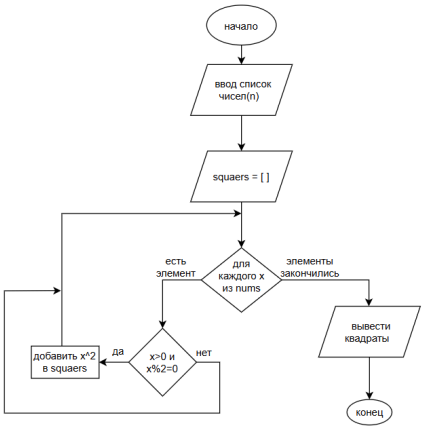


Рисунок 3 – схема программы для второго задания

Алгоритм: Если $n < 2$ — «Не простое». Иначе перебирать делители от 2 до <= n. При нахождении делителя — «Не простое», иначе — «Простое».

Изображение результата выполнения программы 4.

```
n = int(input())

if n < 2:
    print("Не простое")
else:
    is_prime = True
    d = 2
    while d * d <= n:
        if n % d == 0:
            is_prime = False
            break
        d += 1

    if is_prime:
        print("Простое")
    else:
        print("Не простое")
```

Рисунок 4 – код второго задания

Задание тренирует оптимизированную проверку делимости с ограничением по квадратному корню.

1.3 Задание 3. Разложение на простые множители

Постановка задачи: Пользователь вводит целое число n. Если $n < 2$ — вывести «Ошибка». Иначе разложить число на простые множители и вывести их в порядке неубывания через пробел.

Изображение схемы для решения третьего задания 5.

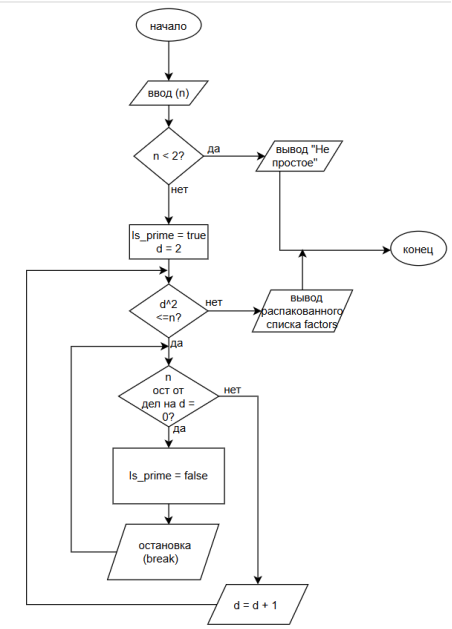


Рисунок 5 – схема программы для третьего задания

Алгоритм: Последовательно делить n на делители, начиная с 2. При делении без остатка выводить делитель и уменьшать n. Повторять проверку того же делителя, пока деление возможно.

Изображение результата выполнения программы 6.

```
n = int(input())

if n < 2:
    print("Ошибка")
else:
    factors = []
    d = 2
    while n > 1:
        while n % d == 0:
            factors.append(d)
            n //= d
        d += 1
    print(*factors)
```

Рисунок 6 – код третьего задания

Задание демонстрирует последовательное разложение числа на простые множители методом пробных делений.

1.4 Задание 4. Цифры числа

Постановка задачи: Пользователь вводит целое число n. Если число отрицательное — вывести «Ошибка». Иначе определить количество цифр и их сумму, работая только с числовым значением (без преобразования в строку).

Изображение схемы для решения четвертого задания 7.

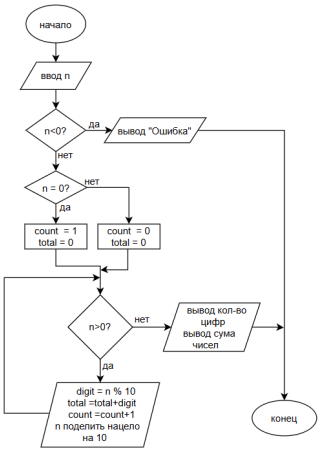


Рисунок 7 – схема программы для четвертого задания

Алгоритм: Пока $n > 0$, выделять последнюю цифру ($nb \bmod 10$), прибавлять её к сумме, увеличивать счётчик и выполнять целочисленное деление n на 10. Для нуля: 1 цифра, сумма 0.

Изображение результата выполнения программы 8.

```
n = int(input())

if n < 0:
    print("Ошибка")
else:
    if n == 0:
        count = 1
        total = 0
    else:
        count = 0
        total = 0
        while n > 0:
            digit = n % 10
            total += digit
            count += 1
            n //= 10

    print(f"Цифр: {count}")
    print(f"Сумма цифр: {total}")
```

Рисунок 8 – код четвертого задания

Задание показывает обработку цифр числа через арифметические операции.

1.5 Задание 5. Таблица расстояний

Постановка задачи: Пользователь вводит целое число n от 1 до 10. При выходе за диапазон — «Ошибка». Построить таблицу расстояний между точками с координатами от 0 до $n - 1$.

Изображение схемы для решения пятого задания 9.

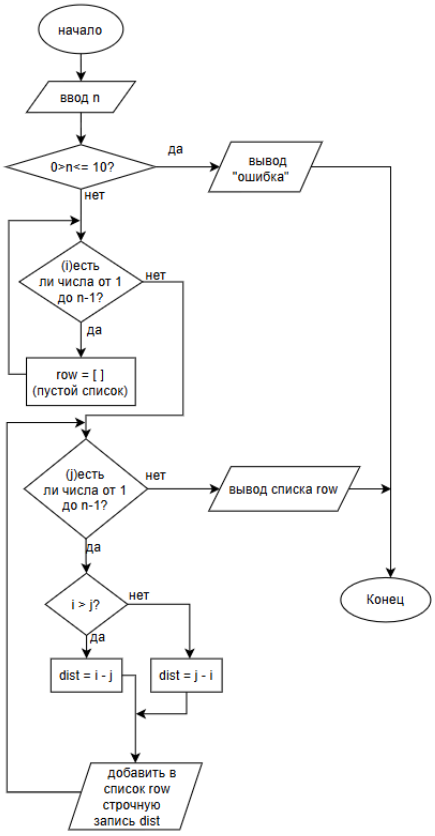


Рисунок 9 – схема программы для пятого задания

Алгоритм: Двумя вложенными циклами перебрать все пары (i, j). Расстояние = $|i - j|$. Вывести строки таблицы без заголовков, элементы через пробел.

Изображение результата выполнения программы 10.

```
1   n = int(input())
2
3   if not 1 <= n <= 10:
4       print("Ошибка")
5   else:
6       for i in range(n):
7           row = []
8           for j in range(n):
9               if i > j:
10                  dist = i - j
11              else:
12                  dist = j - i
13              row.append(str(dist))
14          print(' '.join(row))
```

Рисунок 10 – код пятого задания

Задание тренирует работу с вложенными циклами и построение двумерной таблицы. Основное требование — точное соответствие формату вывода без лишних пробелов и заголовков.